

LayerIt! — Example Sections for Thesis Revision

Illustrative drafts for the merged 6-chapter structure

These are illustrative examples of what the proposed new structure looks like in practice — not finished text. Some were written from ASP's description of the intended framing; in others the existing content was reworked into the new shape. Treat them as models to adapt, not prose to paste verbatim.

Shaded, blue-ruled blocks = draft prose intended for the thesis. **Amber NOTE boxes** = ASP's guidance, not for the thesis.

Part 1 — Objectives, RQs & Contributions (§1.3) + Achievements (§6.1)

Part 1A — Objectives / RQ / Contributions block (new §1.3)

1.3 Objectives, Research Questions and Contributions

The overall goal of this thesis is to develop and assess a modular, object-oriented Python framework for composing time-aligned, multi-modal MIR visualizations — combining audio representations, symbolic scores, and algorithmic outputs on a shared performance timeline — that are otherwise achievable only through manual, non-reproducible image editing. The contribution is not the alignment itself (which the framework obtains through ScoreWarp), but the object-oriented *layer-composition* model that makes such figures expressible in code, and the demonstration of the figures and workflows this enables.

We pursue three objectives:

- O1.** Develop a reusable Python framework for composing score-aligned MIR visualizations.
- O2.** Demonstrate that the framework supports visualization workflows that are difficult or impractical with current MIR tools.
- O3.** Assess the usefulness of the framework for beat annotation and beat-tracking evaluation workflows.

These objectives are examined through three research questions:

- RQ1.** How can heterogeneous MIR visualizations be combined into a common score-aligned representation? (→ O1)
- RQ2.** To what extent does LayerIt simplify the creation of score-aligned visualizations compared with manual SVG editing, Sonic Visualiser, and Piano Precision — and how readily can new visualizations be added? (→ O2)
- RQ3.** How can LayerIt's score-aligned visualizations support beat annotation and beat-tracking evaluation, and under what conditions does that support break down? (→ O3)

RQ1 is answered by the framework's design (Chapter 4); RQ2 and RQ3 are answered in the evaluation (Chapter 5). Each research question is phrased to elicit a graded, evidence-based answer rather than a yes/no verdict, and the limits of the approach are examined within RQ2 (where simplification ceases) and RQ3 (where annotation/evaluation support breaks down) rather than as a separate question.

Contributions. Beyond the dissertation itself, this work delivers: (i) **LayerIt**, the object-oriented visualization framework; (ii) its release as an **open-source, pip-installable library**; (iii) an **open-source repository** documenting the five exploratory prototypes that preceded it; and (iv) a **Late-Breaking/Demo paper submitted to ISMIR 2026** (under review at the time of writing).

NOTE Phrase the ISMIR item precisely — “submitted / under review,” never “accepted” or “published.” A Late-Breaking/Demo is a real but modest, typically non-archival signal: present it as dissemination and community engagement, not as peer-reviewed validation of the research claims.

NOTE RQ3's “...under what conditions does that support break down?” commits Fernando to actually running an expressive/rubato example (Section 5.5). Keep the clause only if those cases will exist.

Part 1B — Rewritten §6.1 “Achievements” — closing on the objectives O1–O3

The current Achievements section (now §6.1, since Conclusions becomes Chapter 6) lists seven achievements. Restructure so the section closes on the three objectives — did we achieve O1, O2, O3? — each citing its RQ's evidence in Chapter 5, then lists the delivered contributions. Draft:

6.1 Achievements

This thesis set out to develop and assess a modular, object-oriented framework for composing score-aligned MIR visualizations. We review each objective in turn.

O1 — A reusable framework (RQ1). We designed and implemented LayerIt, whose object-oriented “layers within layers” model combines heterogeneous visualizations — audio representations, symbolic score, and algorithmic outputs — into a common score-aligned representation (Chapter 4). The model is realized consistently at the visual, SVG, and code levels, and the framework is released as an open-source, pip-installable library, making it directly reusable by the community.

O2 — Workflows difficult with current tools (RQ2). In Sections 5.2–5.4 we demonstrated, from our own data, composite figure types that previously appeared only as hand-assembled images, and compared LayerIt against manual SVG editing, Sonic Visualiser, and Piano Precision. LayerIt is the only workflow that places a time-warped score on the same axis as arbitrary algorithmic curves while remaining fully scriptable and reproducible; adding a new visualization required only [N] lines implementing the shared Layer interface, with no change to the core. We also report where the simplification has limits (Section 5.5).

O3 — Usefulness for annotation and evaluation (RQ3). In Section 5.3 we showed how a score-aligned view supports beat-tracking evaluation — contrasting discrete beat estimates with the underlying probability curve, an inspection Sonic Visualiser cannot reproduce in score-aligned form — and argued that, for annotation, a time-warped score beneath the spectrogram serves the annotator better than link-based connecting lines. Section 5.5 documents the conditions under which this support degrades: highly expressive performances that depart from the notated score, and sparse onset annotation. [SUMMARIZE ACTUAL FINDING.]

Beyond these objectives, the work delivers three further contributions:

1. **An open-source, pip-installable release** of LayerIt, lowering the barrier to community adoption and extension.
2. **An open-source repository of the five prototypes** (Chapter 3), documenting the exploratory path that separated the interaction and visualization problems and fixed the thesis scope.
3. **A Late-Breaking/Demo paper submitted to ISMIR 2026** (under review), disseminating the work to the research community.

NOTE Fill [N] and the RQ3 finding once the evaluation work is done. §6.1 now closes on objectives; the RQ evidence is cited within each objective.

Part 2 — Methodology Chapter (draft)

Standalone Chapter 3, placed before the merged “LayerIt: Design and Implementation” chapter (Ch. 4). The iterations currently in the draft’s §3.2 move here and are reframed as design cycles. The key additions over the current text are: (a) naming the research approach, (b) reframing each iteration as a cycle with an explicit lesson carried forward, (c) recording the scope decision, and (d) stating the evaluation criteria up front so the Evaluation chapter can operationalize them.

Chapter 3 — Methodology

This chapter describes the research approach taken in this thesis. Rather than following a single linear specification, the work proceeded through an iterative, exploratory process in which successive prototypes were built, evaluated, and discarded or carried forward. We first situate this approach, then trace the five prototype iterations as a sequence of design cycles, record the central scope decision that emerged from them, and finally state the criteria against which the resulting framework is evaluated in Chapter 5.

3.1 Research Approach

This thesis follows an exploratory, iterative *research-through-design* approach. The artifact developed — LayerIt — is both the outcome of the research and the instrument through which the research problem was understood. Our understanding of *what* visualization problem most needed solving was not fixed at the outset; it was refined through the act of building and evaluating prototypes. Early iterations targeted interactive beat-annotation interfaces; only through their construction did it become clear that the underserved problem was visualization rather than interaction, and that a reusable, code-based tool would offer the community more value than another GUI.

This is a recognized mode of inquiry for systems and tooling work, where the contribution is an artifact and a set of design principles rather than an empirical hypothesis test. Each iteration was treated as a design cycle: a goal or hypothesis, a prototype built to pursue it, an evaluation of what the prototype revealed, and a design decision carried into the next cycle.

3.2 Iterative Development Process

The development of LayerIt passed through five prototype iterations before the final framework took shape. Each is summarized below as a cycle, emphasizing the lesson that shaped the eventual direction. Table 3.1 consolidates the sequence.

Iteration 1 — BeatMapJS. *Goal:* explore interactive, web-based beat-annotation features. *Built:* a JS/HTML interface (BeatThis, Wavesurfer.js) with an interactive spectrogram, region annotation, a tempo-annotation table, and SV-compatible .txt import/export. *Lesson:* the prototype added no annotation feature beyond BeatThis integration, and — critically — produced no reusable Python library, which we had identified as essential for community uptake. *Decision carried forward:* a reusable Python core is a hard requirement.

Iteration 2 — MEI-Basic-Edit. *Goal:* let users manually align score elements against an audio layer, testing Audio-to-Score Alignment as an annotation aid. *Built:* a JS/HTML/Verovio tool rendering an MEI score below a static spectrogram into a single SVG, with drag-to-align interaction and an embedded XML editor. *Lesson:* interactive score editing is costly in both UI/UX and engineering terms; but the work yielded deep familiarity with MEI, Verovio, and SVG structure that proved essential later. *Decision carried forward:* SVG is a viable composition substrate; interactive editing is not where the value lies.

Iteration 3 — BeatPrecision, a Piano Precision fork. *Goal:* add beat-annotation features on top of PP's existing score alignment and SV functionality. *Built:* a Qt/C++ fork integrating BeatThis with horizontal score follow-along. *Lesson:* the SV/PP C++ codebase was large, slow to compile, and hard to extend; more importantly, we could not solve interaction and visualization simultaneously and had to choose. We chose visualization, seeing the clearer community gap. *Decision carried forward:* separate the interaction problem from the visualization problem; pursue visualization.

Iteration 4 — Score-Alignment.py. *Goal:* build score-aligned spectrograms in Python, exploring the Modusa framework. *Built:* an interactive matplotlib/Modusa spectrogram. *Lesson:* a genuinely fluid interactive spectrogram (zoom, pan, playback cursor) was far harder than expected and the result was unsatisfactory. *Decision carried forward:* abandon real-time interactivity; favour static, high-quality, reproducible output.

Iteration 5 — Score-Alignment.js. *Goal:* re-attempt the aligned-spectrogram tool in JS/HTML with richer score interaction. *Built:* a Verovio/MEI/Wavesurfer.js system storing beat timestamps in the MEI <when> element with note highlighting. *Lesson:* promising, but it reinforced the need for a *strictly visualization-focused, Python, object-oriented* tool that the community could reuse and extend. *Decision carried forward:* this defined the brief for LayerIt.

NOTE Dates are deliberately omitted — this is a dissertation, not a progress report, so when each iteration happened or how long it took carries no analytical weight. What matters is the sequence of lessons, not the calendar.

Table 3.1 — Summary of development iterations

#	Iteration	Technologies	Key lesson	Decision carried forward
1	BeatMapJS	JS/HTML, BeatThis, Wavesurfer.js	No reusable Python library produced	Python core is mandatory
2	MEI-Basic-Edit	JS/HTML, Verovio, MEI, SVG	Interactive score editing is costly; SVG mastery gained	SVG as composition substrate
3	BeatPrecision (PP fork)	Qt/C++, BeatThis	SV/PP codebase hard to extend; can't do interaction + visualization at once	Separate the two; pursue visualization
4	Score-Alignment.py	Python, Modusa, matplotlib	Fluid interactivity impractical	Static, reproducible output
5	Score-Alignment.js	JS/HTML, Verovio/MEI, Wavesurfer.js	Need a Python, OO, visualization-only tool	Defined the LayerIt brief

3.3 Scope Decision

The iterations above record a deliberate narrowing of scope: from an *interactive beat-annotation GUI* to a *reusable, object-oriented visualization library*. Two observations drove this. First, interaction and visualization are distinct problems that could not both be solved within a single thesis; attempting both produced prototypes that were weak at each. Second, visualization — specifically, time-aligned visualization combining score, audio, and algorithmic data — was the more under-served gap, where existing GUI tools could not keep pace with research needs and where researchers fell back on ad hoc manual editing. Focusing on visualization, and delivering it as reusable Python rather than another GUI, was judged to offer the community the greatest value.

3.4 Evaluation Criteria

Because the contribution is an artifact and a design approach rather than an empirical claim, and because a controlled user study was not feasible within the project's scope, LayerIt is evaluated qualitatively, through demonstration and structured comparison. We assess it against five criteria, applied in Chapter 5:

1. **Coverage / expressiveness** — can the layer model express the composite, score-aligned figure types found in the literature? (RQ1 / O2)
2. **Manual-effort reduction** — how much manual work does a LayerIt workflow save relative to manual SVG editing and to GUI tools, including time to generate a figure? (RQ2 / O2)
3. **Reproducibility** — can a figure be regenerated automatically when the underlying data changes? (RQ2 / O2)
4. **Extensibility** — what is the cost, in code and in core modifications, of adding a new visualization type? (RQ2 / O2)
5. **Fidelity and limits** — under what conditions does the approach degrade? (RQ2, RQ3 / O2, O3)

Defining these criteria here allows the evaluation that follows to be read as a deliberate assessment against stated goals rather than an open-ended demonstration.

Part 3 — Evaluation Chapter (skeleton + filled tables + figure placeholders)

New chapter, placed after the merged “LayerIt: Design and Implementation” chapter (Ch. 4) and before “Conclusions and Future Work” (Ch. 6). Built as structured qualitative case studies, since no user study was possible. The criteria defined in §3.4 are operationalized here.

Chapter 5 — Evaluation

This chapter evaluates LayerIt against the criteria defined in Section 3.4. As a controlled user study was outside the scope of this project, we evaluate through three complementary means: a capability demonstration that reproduces, from our own data, composite figure types previously dependent on manual editing (Section 5.2, RQ1); a structured comparison of annotation and evaluation workflows against existing tools (Section 5.3, RQ2 and RQ3); an extensibility probe measuring the cost of adding a new layer (Section 5.4, part of RQ2); and a documented analysis of the framework's limits on difficult material (Section 5.5, closing RQ2 and RQ3).

5.1 Evaluation Approach

We assess LayerIt qualitatively, through demonstration and structured comparison, against the five criteria of Section 3.4: coverage, manual-effort reduction, reproducibility, extensibility, and fidelity/limits. Two methodological points should be made explicit. First, the figures in this chapter are **generated from our own source data** (audio, MEI score, MAPS, and BeatThis output) rather than copied from published work; published figures are referenced only to motivate the figure *types* we target. Second, where we report effort or time, we separate **one-time setup cost** (obtaining the MEI score, MAPS file, and warped score) from **per-figure cost**, since the two behave very differently across workflows.

NOTE The “generated from our own data” point is essential. It is both a stronger contribution and the way to avoid reproducing copyrighted figures. One referenced example (Rev98) is explicitly unpublished and must NOT be shown or reproduced — pursue the figure type from own data only.

5.2 Reproducing Composite Figure Types (RQ1)

The MIR literature is full of composite, time-aligned figures whose construction required manual image assembly because no tool could place a time-warped score on the same axis as audio and algorithmic data. This section demonstrates that LayerIt can produce equivalent figures programmatically. We organize the demonstration around four figure types drawn from the literature, reproducing each from our own data.

5.2.1 Score + audio + chroma on a shared axis. Textbook multimodal alignment (cf. Müller, *FMP*, Fig. 5.1 [10]; Ewert et al. on score-informed editing [7]) combines a score, a waveform or spectrogram, and a derived representation such as a chromagram or chord-region strip on one time axis. Reproducing this from our own data shows the layer model generalizes beyond beat tracking. It also illustrates the *region* idea noted by Ewert et al.: because all layers share one timeline, a marked region projects identically across every layer.

Figure [5.1]: LayerIt-generated score + spectrogram + chromagram on a shared timeline, from [DATASET/PIECE]. [PLACEHOLDER — Fernando to generate.]

5.2.2 Score-aligned beat-tracking analysis (the gap case). The clearest “fills a real gap” example is Sá Pinto et al. [SHIFT-IF-YOU-CAN], whose beat-tracking figure can only be understood musically *together with a separate score figure* carrying bar numbers, precisely because the authors could not align the score to the time-based plot. LayerIt collapses these into one figure: the time-warped score sits directly above the beat-tracking layers, making bar context immediately legible.

Figure [5.2]: A single LayerIt figure combining time-warped score with beat-tracking output, replacing the two-figure arrangement otherwise required. [PLACEHOLDER.]

5.2.3 Stacked probability-and-position figure. A common evaluation figure stacks, per subplot, the beat and downbeat novelty/probability curves (overlaid on shared axes) above the discrete beat and downbeat positions. We construct an equivalent figure from BeatThis output with LayerIt.

Figure [5.3]: LayerIt-generated stack of beat/downbeat probability curves and discrete positions, from our own BeatThis run. [PLACEHOLDER.]

NOTE This is the type exemplified by the unpublished Rev98 figure. Build it from own data; do NOT reproduce the Rev98 image.

5.2.4 Layered time-based display. A simpler but ubiquitous type (cf. Morais et al. [SKIP-THAT-BEAT], Fig. 1) layers a waveform with beat positions across processing steps. We reproduce it to show the model handles the lightweight end of the spectrum as cleanly as the complex end.

Figure [5.4]: Waveform + beat-position layers. [PLACEHOLDER.]

Coverage summary. Table 5.1 maps each literature figure type to the LayerIt layers used and notes whether the figure previously required manual editing.

Table 5.1 — Literature figure types reproduced with LayerIt

Figure type	Literature exemplar	Layers used	Previously required
Score + audio + chroma	Müller FMP 5.1 [10]; Ewert et al. [7]	Score, spectrogram/waveform, chroma	Manual assembly
Score-aligned BT analysis	Sá Pinto et al.	Score, spectrogram, beat/downbeat, onsets	Two separate figures
Probability + positions stack	(unpublished; type only)	Beat/downbeat prob., beat/downbeat positions	Manual assembly
Layered time-based display	Morais et al.	Waveform, beat positions	Manual assembly / scripting

5.3 Workflow Comparison: Annotation and Evaluation (RQ2 + RQ3)

We compare LayerIt to Sonic Visualiser (SV), Piano Precision (PP), and manual SVG editing across the two task families that motivated the thesis.

5.3.1 Evaluation workflow. Beat-tracking evaluation benefits from seeing not just an algorithm's discrete beat estimates but its underlying confidence. Figure [ref to the former Fig. 4.2, promoted here] shows BeatThis's discrete output above its frame-by-frame probability curve, both aligned to the score. The probability view reveals beats the discrete output misses but where the model in fact shows clear probability peaks — an evaluation insight that SV cannot present in score-aligned form, and that PP cannot present because its alignment is link-based and it cannot overlay arbitrary algorithmic curves.

5.3.2 Annotation workflow. For annotation, the key requirement is that the annotator can read the score *against the time axis* while placing or correcting beats. Here the distinction between time-warped and link-based alignment is decisive: PP's link-based connecting lines, useful for navigation, do not put the score physically under the time axis, which is what the annotator needs to judge whether a beat falls on the notated event. LayerIt's time-warped score does exactly this.

NOTE Support this with a concrete walkthrough/figure of annotating one excerpt; even a qualitative step-by-step is persuasive.

5.3.3 Feature and effort comparison. Table 5.2 compares the four workflows on the five evaluation criteria.

Table 5.2 — Workflow comparison

Dimension	Sonic Visualiser	Piano Precision	Manual SVG editing	LayerIt
Score on shared time axis	X	Link-based only	✓ (by hand)	✓ (time-warped)
Overlay arbitrary algorithmic curves	Limited	X	✓ (by hand)	✓
Reproducible / scriptable	Partial	X	X	✓
Effort to regenerate after data change	Redo manually	n/a	Redo by hand	Re-run script
Add a new visualization type	Plugin (C++)	Plugin (C++)	n/a	Subclass Layer

5.3.4 Effort to generate a figure. To make the effort dimension concrete, we counted the **manual operations** (steps requiring human judgement) needed to produce [2–3] representative figures, both by the manual SVG-editing process used before LayerIt and by the LayerIt script, separating one-time setup from per-figure cost. Table 5.3 reports the results.

Table 5.3 — Effort to produce representative figures (illustrative structure — fill with measured values)

Figure	Manual SVG editing (# ops)	LayerIt — first figure (# ops, incl. setup)	LayerIt — each variant after (# ops)
[Fig A]	[# ops]	[# ops]	[# ops]
[Fig B]	[# ops]	[# ops]	[# ops]

The expected and defensible pattern is that the first LayerIt figure is comparable to manual editing once setup is counted, while every subsequent variant costs a fraction of the operations and is reproducible.

NOTE A manual-operation count is the recommended metric — it is low-noise and captures “less work” directly. Fernando may substitute or add wall-clock time if that reads as more convincing, but should report honestly and not inflate either way.

5.4 Extensibility Probe (RQ2)

To assess extensibility concretely, we implemented a new layer type not present in the original framework — [e.g. a waveform layer, or an onset-novelty layer matching the Morais example]. Adding it required implementing only the two methods of the shared Layer interface (`load_data` and `draw`) in [N] lines of code, with no modification to the Visualizer orchestration logic. This confirms the modular-composability claim of Chapter 4: new visualization types are self-contained modules that compose with existing layers without touching the core.

NOTE Do this for real and report the actual line count and the diff scope. “Added layer X in N lines without modifying the core” is a checkable result.

5.5 Limitations and Difficult Cases (RQ2 + RQ3)

The approach assumes a reasonably faithful correspondence between score and performance. We document two stress cases.

5.5.1 Highly expressive performance. We ran [EXPRESSIVE/RUBATO PIECE] through the pipeline, where the performer departs substantially from the notated score in tempo and timing. [DESCRIBE WHAT HAPPENS: where time-warping the score becomes unreliable; how misalignment manifests visually; whether it degrades gracefully or breaks.] This is a limitation of the underlying alignment (ScoreWarp/MAPS), not of the layer-composition model, but it bounds where LayerIt is currently useful.

Figure [5.5]: Degradation under an expressive performance. [PLACEHOLDER.]

5.5.2 Sparse onset annotation. ScoreWarp interpolates between available onset anchors, so sparse MAPS data trades alignment precision for annotation effort. We illustrate the resulting interpolation error with [DOWNBEAT-ONLY vs. DENSE] annotation on the same excerpt.

5.5.3 Final-SVG hand-editing as a fallback. LayerIt’s SVG output can be edited by hand as a last resort for cases the pipeline does not yet cover. We note explicitly that this is a fallback, not a feature: hand-editing sacrifices the reproducibility that is the framework’s central advantage, and reintroduces exactly the manual, non-reproducible process LayerIt was built to replace.

5.6 Chapter Summary

[Summarize by RQ: RQ1 — the layer model combines heterogeneous visualizations into one score-aligned representation, reproducing the principal literature figure types from data; RQ2 — LayerIt uniquely combines a time-warped score with arbitrary curves while staying reproducible, saving manual effort on every figure after the first, and new layers cost $\sim[N]$ lines with no core changes; RQ3 — score-aligned views support beat annotation and evaluation, degrading on highly expressive material and sparse anchors.]